

TOMATO LEAF DISEASE DETECTION USING CNN

Mini Project Document

ABSTRACT

Tomato plants can be affected by diseases that appear as visible changes on their leaves. Identifying these symptoms manually can take time and may be difficult when many plants need to be checked. This mini project explores an image-classification system that uses a Convolutional Neural Network (CNN) to classify tomato leaf images into the categories represented in its training dataset. The application is intended to provide a quick preliminary prediction from an uploaded image. Python and TensorFlow/Keras are used for model development, while Streamlit provides a simple user interface. The project demonstrates how deep learning and computer vision can support plant-health monitoring. Predictions should be treated as decision-support information rather than a guaranteed diagnosis, because image quality, lighting, dataset coverage, and real-world conditions can affect performance.

1. Abstract

Tomato plants can be affected by diseases that appear as visible changes on their leaves. Identifying these symptoms manually can take time and may be difficult when many plants need to be checked. This mini project explores an image-classification system that uses a Convolutional Neural Network (CNN) to classify tomato leaf images into the categories represented in its training dataset. The application is intended to provide a quick preliminary prediction from an uploaded image. Python and TensorFlow/Keras are used for model development, while Streamlit provides a simple user interface. The project demonstrates how deep learning and computer vision can support plant-health monitoring. Predictions should be treated as decision-support information rather than a guaranteed diagnosis, because image quality, lighting, dataset coverage, and real-world conditions can affect performance.

2. Introduction

Tomato is an important crop, and leaf diseases can reduce plant health and productivity. Early identification may help growers decide when to inspect plants more closely or seek agricultural advice. Computer vision makes it possible to analyse leaf images automatically. In this project, a CNN learns visual patterns from labelled examples and uses those patterns to predict a class for a new image.

The system follows a basic pipeline: collect labelled leaf images, prepare and resize the images, train or load a CNN model, accept an image through the interface, preprocess it in the same way as the model expects, and display the predicted class.

3. Problem Statement

Manual identification of tomato leaf diseases can be slow and depends on the observer’s experience. The project addresses the need for a simple image-based tool that can provide a rapid, consistent preliminary classification of tomato leaf images.

4. Objectives

• Build an image-classification workflow for tomato leaf images.

• Use a CNN-based deep-learning model to learn visual features.

• Provide a simple interface for uploading an image and viewing a prediction.

• Evaluate model performance using a validation or test dataset.

• Document limitations and identify improvements for practical use.

5. Scope and Dataset

The system is intended for tomato leaf images and can classify only the categories included in the dataset used to train the model. Earlier project notes identify healthy leaves, early blight, and late blight as classes of interest. Other classes must not be claimed unless confirmed in the actual dataset and model configuration.

Dataset details to confirm before submission: dataset source and licence; exact class names; number of images per class; training/validation/test split; and whether images from the same plant or source appear across splits. A balanced, well-separated dataset helps produce a more meaningful evaluation.

6. Technologies Used

Technology	Purpose
Python	Main programming language.
TensorFlow / Keras	Build, train, load, and run the deep-learning model.
CNN	Learn spatial and visual patterns from leaf images.
Streamlit	Create a browser-based interface for image upload and prediction.
Pillow (PIL)	Open and prepare image files.
NumPy	Represent image data as numerical arrays.

7. System Architecture

Input image → Image preprocessing → Trained CNN model → Predicted class → Streamlit results screen

The image is opened and converted to the format expected by the model. It is resized to the required input dimensions and normalized if the training pipeline uses normalization. The processed image is passed to the loaded model. The model outputs scores or probabilities for its learned classes; the application selects the highest-scoring class and displays it. Preprocessing and class-index mapping in the application must match those used during training.

8. Methodology

8.1 Data preparation

Load labelled tomato leaf images, check for corrupt or duplicate files, resize them to the model's input dimensions, and apply the same normalization during training and inference. If augmentation is used, apply it only to training images.

8.2 Model training

A CNN typically learns low-level features such as edges and textures in early layers and more complex patterns in deeper layers. The training process adjusts model parameters to reduce classification error on the training data. Validation data can be used to monitor generalization and select model settings.

8.3 Prediction

When a user uploads a leaf image, the application preprocesses it, runs inference with the trained model, maps the output index to the correct class label, and shows the result.

8.4 Evaluation

Evaluate on a held-out test set. Report accuracy together with per-class precision, recall, F1-score, and a confusion matrix. Validation accuracy reported during development was 85.61%; this is a historical validation result, not a verified final test accuracy. Re-run the final evaluation and insert the resulting metrics before submission.

9. Implementation Reference

The earlier working setup used Python 3.11.15, TensorFlow 2.21.0, Streamlit 1.55.0, NumPy 2.4.3, and Pillow 12.1.1. The working folder was named **tomato_pj**, with a Streamlit interface in **app.py** and a saved model named **tomato_model.h5**. Confirm that these files and versions match the current working project.

Example inference flow (illustrative)

```
from PIL import Image
import numpy as np
```

```
import streamlit as st
from tensorflow.keras.models import load_model

model = load_model("tomato_model.h5")
class_names = ["Early Blight", "Healthy", "Late Blight"]  # Verify order!

uploaded = st.file_uploader("Upload a tomato leaf image",
                            type=["jpg", "jpeg", "png"])
if uploaded is not None:
    image = Image.open(uploaded).convert("RGB")
    st.image(image, caption="Uploaded leaf", use_container_width=True)
    # Resize and normalize exactly as in model training.
    image = image.resize((IMG_WIDTH, IMG_HEIGHT))
    array = np.asarray(image, dtype=np.float32) / 255.0
    array = np.expand_dims(array, axis=0)
    scores = model.predict(array)
    predicted_index = int(np.argmax(scores, axis=1)[0])
    st.write("Prediction:", class_names[predicted_index])
```

Important: This is a template, not a guaranteed drop-in replacement for the existing app. Set IMG_WIDTH and IMG_HEIGHT to the actual model input size, confirm whether division by 255 matches training, and copy the exact class order from the training code.

10. Results and Discussion

The previous development notes record a validation accuracy of 85.61%. This suggests the model learned useful patterns on its validation data, but it does not establish the same accuracy for images from farms or phones under different conditions. The final report should include the current training history, test-set metrics, confusion matrix, and representative correct and incorrect predictions. Do not describe unmeasured performance as a confirmed result.

11. Advantages

• Provides a quick preliminary prediction from a leaf image.

• Offers a simple interface that does not require users to run training code.

• Demonstrates a practical application of CNNs and computer vision.

• Can be extended with more data, additional classes, and improved evaluation.

12. Limitations

• The model can only predict classes represented in its training labels.

• Poor lighting, blurry images, backgrounds, and unfamiliar symptoms can reduce reliability.

• Dataset bias or leakage can make evaluation results overly optimistic.

• The prediction is not a substitute for expert agricultural diagnosis.

13. Future Enhancements

• Add a verified test-set evaluation and per-class performance dashboard.

• Improve data diversity using images from different devices, lighting conditions, and farms.

• Display confidence scores with a caution message for uncertain predictions.

• Add an image-quality check and an option to flag images outside the known classes.

• Explore transfer learning models such as MobileNetV2, DenseNet201, or InceptionV3 and compare them using the same data split.

• Deploy the app with suitable privacy, reliability, and maintenance practices.

14. Conclusion

This mini project demonstrates a CNN-based approach to classifying tomato leaf images through a simple Streamlit interface. The recorded validation accuracy from earlier development was 85.61%, but final performance must be confirmed using a reproducible held-out test set. With better dataset diversity, careful evaluation, and a consistent preprocessing pipeline, the project can be improved as a useful educational prototype for preliminary plant-health screening.

15. References

• TensorFlow documentation: https://www.tensorflow.org/

• Streamlit documentation: https://docs.streamlit.io/

• scikit-learn documentation for classification metrics: https://scikit-learn.org/stable/modules/model_evaluation.html

• Pillow documentation: https://pillow.readthedocs.io/

Before submission: Confirm your institution name, department, guide name, exact dataset classes and source, model input dimensions, class-index order, final test metrics, and any required certificate/declaration pages. The illustrative code and reference details should be reconciled with the actual project files.